

Estratégias de Recuperação em RAG

Além da busca vetorial: um guia prático

Laboratórios **embeddings-neo4j** e **titanic-graphrag**

Engenharia de IA Aplicada

Setembro de 2026

Sumário

1. Por que a busca vetorial não basta

1. O que um embedding realmente mede
2. Uma medição no acervo do Titanic

2. As que faltam

1. Busca lexical (BM25)
2. Busca híbrida
3. MMR — diversidade nos resultados
4. Reranking

3. As mais sofisticadas

1. Small-to-big (Parent Document)
2. Self-query
3. HyDE
4. Multi-vetor e late interaction (ColBERT)

4. Como escolher

1. Por que a busca vetorial não basta

1.1 O que um embedding realmente mede

Um modelo de embeddings transforma texto num vetor de números — 384 dimensões, no caso do `paraphrase-multilingual-MiniLM-L12-v2` usado nos laboratórios. Textos com sentido parecido ficam próximos nesse espaço; a busca consiste em achar os `k` vizinhos mais próximos da pergunta.

Isso é **parecença**, não verdade nem filtro. A operação toda tem três passos: vetorizar, medir distância, ordenar. Não existe nenhum ponto em que o sistema conte, agrupe, verifique um predicado ou garanta exatidão.

A consequência prática

Busca vetorial nunca produz uma agregação. Ela devolve `k` documentos escolhidos por proximidade — e `k` é um número fixo, definido antes da pergunta existir.

1.2 Uma medição no acervo do Titanic

Para tornar isso concreto, os 891 passageiros do dataset do Kaggle foram convertidos em frases e embedados como se fossem documentos. A pergunta: "*Quantas mulheres da terceira classe sobreviveram?*". A resposta correta é **72**.

topK	corretos	errados	precisão
3	3	0	100%
10	8	2	80%
50	34	16	68%
72	41	31	57%
200	68	132	34%

Nos primeiros resultados a busca acerta bem — ela é competente em achar quem *se parece* com a pergunta. Mas para capturar as 72 corretas seria preciso ler os **328 primeiros resultados**, arrastando 256 passageiros errados. Não existe corte que separe limpo, porque a similaridade decai suavemente: ela nunca soube o que é "mulher da terceira classe", só o que se parece com a frase da pergunta.

Daí a divisão que os laboratórios adotam: perguntas com predicado e agregação vão para o Cypher; perguntas sobre significado vão para os embeddings. Este guia trata das estratégias que ficam *entre* esses dois extremos.

2. As que faltam

2.1 Busca lexical (BM25)

Ideia	casar termos exatos, ponderados pela raridade
Resolve	nomes próprios, códigos, siglas, jargão
Custo	baixíssimo — índice invertido, sem GPU
No projeto	índice full-text do Neo4j

BM25 (*Best Matching 25*) é o algoritmo clássico de recuperação por palavra-chave, herdeiro do TF-IDF. Ele pontua um documento por três fatores:

- **Frequência do termo** — quantas vezes a palavra aparece no documento, com retorno decrescente: a décima ocorrência vale muito menos que a segunda.
- **Raridade do termo** — uma palavra que aparece em poucos documentos discrimina mais. "Carpathia" vale muito; "navio" vale pouco.
- **Tamanho do documento** — normaliza para que documentos longos não levem vantagem por acaso.

É o oposto complementar do vetorial. Onde os embeddings brilham — sinonímia, paráfrase, tradução — o BM25 falha. E onde o BM25 brilha — um código de bilhete como CA. 2343, um nome como *Carpathia*, uma sigla — os embeddings falham, porque esses tokens não têm semântica a capturar.

O ponto cego atual do laboratório

Uma pergunta como "o que dizem sobre o *Carpathia*?" depende hoje de sorte semântica. O nome do navio de resgate não tem significado que o embedding represente bem — ele é apenas uma sequência de tokens raros.

2.2 Busca híbrida

Ideia	rodar vetorial e lexical, fundir os rankings
Resolve	o ponto cego de cada uma das duas
Custo	baixo — duas buscas em vez de uma
No projeto	searchType: "hybrid" — uma palavra

É o padrão de produção hoje. Cada mecanismo cobre a fraqueza do outro, e a única questão é como combinar duas listas ordenadas por escalas incomparáveis: similaridade de cosseno vai de 0 a 1, BM25 não tem teto.

A resposta usual é **Reciprocal Rank Fusion** (RRF), que ignora os scores e usa apenas as posições:

$$\text{score}(d) = \sum_i \frac{1}{(k + \text{posição de } d \text{ na lista } i)}$$

com $k \approx 60$

Um documento que aparece em 3º lugar nas duas listas vence um que foi 1º em uma e não apareceu na outra. A constante `k` amortece o peso das primeiras posições, evitando que um único ranking domine.

No `Neo4jVectorStore` a mudança é mínima:

```
// src/config.ts
searchType: "hybrid" as const, // era "vector"
keywordIndexName: "trechos_keyword",
```

O driver passa a criar um índice full-text além do vetorial e a combinar os resultados. Vale conferir como ele funde os rankings — a implementação é simples e pode não usar RRF.

2.3 MMR — diversidade nos resultados

Ideia	resultados relevantes <i>e diferentes entre si</i>
Resolve	top-k com três variações do mesmo trecho
Custo	baixo — cálculo sobre vetores já recuperados
No projeto	exige implementação manual

Com `chunkOverlap: 200`, trechos vizinhos compartilham 200 caracteres. Um top-3 pode trazer praticamente o mesmo conteúdo três vezes, desperdiçando o orçamento de contexto da LLM e escondendo perspectivas alternativas.

Maximal Marginal Relevance escolhe iterativamente, penalizando quem se parece demais com o que já foi escolhido:

```
MMR = argmax [ λ · sim(d, pergunta) - (1-λ) · max sim(d, d_já_escolhido) ]
           d                                d_já

λ = 1 → só relevância (equivale à busca comum)
λ = 0,5 → equilíbrio usual
λ = 0 → só diversidade
```

O procedimento: recupere um top-20, escolha o primeiro por relevância pura, e vá acrescentando o candidato que maximiza a fórmula até ter 3.

Atenção na implementação

O `Neo4jVectorStore` não implementa `maxMarginalRelevanceSearch`. Seria preciso recuperar os vetores dos candidatos e aplicar o algoritmo manualmente.

2.4 Reranking

Ideia	reavaliar os candidatos com um modelo mais preciso
Resolve	ordenação ruim entre trechos parecidos
Custo	médio — um modelo por par (pergunta, trecho)
No projeto	Xenova/ms-marco-MiniLM-L-6-v2, local

A diferença técnica é a arquitetura. A busca vetorial usa um **bi-encoder**: pergunta e documento são vetorizados *separadamente*, e só no fim se comparam dois vetores. Isso permite pré-calculer todos os embeddings dos documentos — é o que torna a busca rápida — mas o documento nunca "vê" a pergunta.

Um **cross-encoder** recebe os dois juntos, num único texto, e produz uma pontuação de relevância. É muito mais preciso, porque a atenção do modelo relaciona cada palavra da pergunta com cada palavra do trecho. É também caro demais para rodar sobre milhares de documentos — daí a arquitetura em dois estágios:

Busca vetorial/híbrida	→	20 candidatos	(rápido, impreciso)
Cross-encoder	→	reordena os 20	(lento, preciso)
Seleção final	→	3 melhores	

Um caso real do acervo

Para "*O Titanic foi avisado sobre icebergs?*", a busca vetorial devolveu em 2º lugar, com 87,1% — quase empatado com o 1º —, um trecho sobre o valor do seguro pago às vítimas. Ele pontuou alto por compartilhar vocabulário e assunto geral, não por conter a resposta. É exatamente o erro que um reranker corrige.

3. As mais sofisticadas

3.1 Small-to-big (Parent Document)

Ideia	embedar trechos pequenos, devolver o bloco grande
Resolve	o dilema do tamanho do chunk
Custo	médio — exige hierarquia entre trechos
No projeto	natural num banco de grafos

Existe uma tensão que nenhum valor de `chunkSize` resolve:

Trecho pequeno	Trecho grande
Vetor concentrado num assunto só, logo a busca acerta mais. Mas o texto recuperado pode não bastar para responder.	Contexto completo para a LLM responder. Mas o vetor é a média de vários assuntos, e a busca fica difusa.

A saída é desacoplar **o que se busca** do **que se devolve**: indexe trechos de 200 caracteres, e quando um deles for encontrado, entregue à LLM o parágrafo ou a página que o contém.

Em bancos vetoriais comuns isso é um artifício com identificadores de pai guardados nos metadados. Num grafo, é estrutura de primeira classe:

```
(:Trecho {texto, embedding})-[:PARTE_DE]->(:Pagina)-[:PARTE_DE]->(:Documento)
```

```
CALL db.index.vector.queryNodes('trechos_index', 20, $vetorDaPergunta)
YIELD node AS trecho, score
MATCH (trecho)-[:PARTE_DE]->(pagina:Pagina)
RETURN DISTINCT pagina.texto, max(score) AS score
ORDER BY score DESC LIMIT 3
```

É um bom argumento a favor de usar Neo4j como vector store: a hierarquia do documento vira grafo, e a consulta que busca pequeno e devolve grande é uma travessia de uma aresta.

3.2 Self-query

Ideia	a LLM extrai filtros estruturados da pergunta
Resolve	perguntas que misturam filtro e significado
Custo	uma chamada de LLM por pergunta
No projeto	os metadados necessários já existem

Muitas perguntas carregam, implicitamente, um filtro e uma busca semântica ao mesmo tempo:

```
"O que a análise de riscos diz sobre velocidade?"
  ↓
filtro   : fileName = "O Caso Titanic.pdf"
semântica: "velocidade"
```

Sem self-query, a palavra "análise de riscos" entra no vetor da pergunta e apenas *envia* o resultado — sem garantia de que os trechos venham daquele documento. Com self-query, ela vira um `WHERE` que exclui todo o resto.

É o mesmo princípio do *text2cypher* do `titanic-graphrag`, aplicado aos metadados dos trechos em vez das propriedades dos passageiros. Os campos necessários — `fileName`, `pageNumber`, `totalPages` — já são gravados em cada nó.

Num sistema multiempresa, esse filtro deixa de ser conveniência e vira **segurança**: `empresa_id` e `permissao` precisam ser aplicados antes da busca, nunca depois.

3.3 HyDE — Hypothetical Document Embeddings

Ideia	embedar uma resposta inventada, não a pergunta
Resolve	perguntas curtas, vagas ou mal formuladas
Custo	uma chamada de LLM por pergunta
No projeto	+15 s de latência com o modelo gratuito atual

A intuição por trás é uma assimetria fácil de esquecer: **perguntas e documentos não se parecem**. Uma pergunta é curta e interrogativa; um trecho de livro é longo e afirmativo. Ao comparar o vetor de uma com o vetor do outro, compara-se coisas de naturezas diferentes.

O HyDE contorna isso pedindo à LLM que escreva uma resposta plausível — mesmo que inventada — e usando essa resposta como consulta:

Pergunta: "botes?"
↓ LLM redige uma resposta hipotética
"O Titanic dispunha de botes salva-vidas em número insuficiente para todos os passageiros e tripulantes a bordo."
↓ este texto é embedado e usado na busca
Trechos recuperados

Por que funciona mesmo se a resposta inventada estiver errada

Ela não é mostrada a ninguém: serve só como consulta. O que importa é que tenha o *formato* e o *vocabulário* de um trecho de documento, aproximando-se da região certa do espaço vetorial.

3.4 Multi-vetor e late interaction (ColBERT)

Ideia	vários vetores por documento, comparados token a token
Resolve	perda de detalhe do vetor único
Custo	armazenamento uma ordem de grandeza maior
No projeto	estudo, não próximo passo

Resumir um trecho de 1000 caracteres em 384 números descarta muita coisa. Um trecho que fala de velocidade, icebergs e binóculos vira a *média* desses três assuntos — e essa média pode não estar perto de nenhum deles.

O ColBERT guarda **um vetor por token** e, na busca, para cada token da pergunta encontra o token mais parecido do documento, somando esses máximos:

$$\text{score}(\text{pergunta}, \text{doc}) = \sum_{t \in p} \max_{t \in d} \text{sim}(t_{\text{pergunta}}, t_{\text{doc}})$$

Chama-se *late interaction* porque a comparação entre pergunta e documento acontece tarde, no momento da busca — diferente do bi-encoder (que comprime antes) e do cross-encoder (que só funciona sobre poucos candidatos). É um meio-termo: quase a precisão de um cross-encoder, com a possibilidade de pré-computar os vetores.

O preço é o armazenamento: 1000 tokens viram 1000 vetores, não 1. Para 303 trechos isso seria administrável; para um acervo corporativo, é uma decisão de infraestrutura.

4. Como escolher

A pergunta certa não é "qual é a melhor", e sim "qual é o modo de falha que estou vendo".

Sintoma observado	Estratégia indicada
Não acha nomes próprios, códigos ou siglas	Lexical / híbrida
Os três resultados dizem quase a mesma coisa	MMR
O trecho certo aparece, mas em 3º lugar	Reranking
Acha o trecho certo, mas falta contexto para responder	Small-to-big
Traz trechos do documento errado	Self-query
Vai mal com perguntas curtas ou vagas	HyDE
Responde com número errado, mas plausível	Agregação (Cypher), não RAG

Ordem de implementação sugerida

1. **Híbrida** — uma palavra no config, ganho imediato e mensurável.
2. **Reranking** — corrige um problema já demonstrado com dados reais do acervo.
3. **Small-to-big** — resolve o dilema do `chunkSize` de vez.
4. **Self-query** — necessário assim que houver múltiplos documentos ou permissões.
5. O restante, conforme o interesse de estudo.

Meça antes e depois

Toda estratégia deste guia é uma hipótese até ser medida no seu acervo. Monte um conjunto de dez a vinte perguntas com a resposta esperada conhecida, e acompanhe uma métrica simples — em que posição aparece o trecho correto. Sem isso, "melhorar a recuperação" vira opinião.

Documento gerado para os laboratórios **embeddings-neo4j** e **titanic-graphrag**. As medições citadas foram obtidas no próprio acervo do projeto: cinco PDFs sobre o Titanic e os 891 passageiros do conjunto de treino da competição do Kaggle.